

Fórmulas de fuerza

ABS 3.0 — Analysis of Biomechanical Signals

Documentación técnica del módulo `logica/formulas.py` y de su recorrido hasta la gráfica.

Laboratorio de Investigación en Biomecánica y Análisis de Movimiento — UTEC

Contenido

Sección	Qué encontrás
1. Las tres fórmulas	Resumen comparativo en una tabla.
2. Qué hace cada una	Definición, interpretación biomecánica y dónde se dibuja.
3. Funciones de cálculo	Código real, explicado línea por línea.
4. Fórmulas propias	El evaluador de expresiones del usuario y su seguridad.
5. Camino hacia la gráfica	Qué información viaja y qué dibuja cada parte.
6. Agregar una fórmula	Los dos únicos puntos que hay que tocar.

1. Las tres fórmulas

El módulo trae tres fórmulas de fuerza ya escritas y validadas. Las dos primeras combinan las componentes que entrega la plataforma; la tercera reescala cualquier señal para poder compararla entre sujetos.

Fórmula	Expresión	Entradas	Unidad	¿Dibuja curva?	Se muestra en
Fuerza resultante F 	$\sqrt{F_x^2 + F_y^2 + F_z^2}$	Fx, Fy, Fz	N	Sí	Fx, Fy, Fz
Fuerza horizontal Fh 	$\sqrt{F_x^2 + F_y^2}$	Fx, Fy	N	Sí	Fx, Fy, Fz
Fuerza normalizada (peso corporal)	F / (masa · g)	la señal + masa	BW	No	todas las visibles

Ninguna de las tres exige tener un filtro puesto. Un filtro pasa-bajos achata los picos, así que obligarlo haría que todo pico reportado quedara subestimado. El filtro solo pasa a ser imprescindible en fórmulas que derivan (RFD, velocidad de COP), porque la derivación amplifica el ruido.

Dos conceptos que conviene no confundir

Concepto	Significado	Ejemplo en Fh
requiere_ejes	De dónde salen los datos . Define la procedencia que se informa al usuario.	Fx, Fy
mostrar_en	Sobre qué gráficas se dibuja . Puede ser un conjunto más amplio, porque comparar contra otra componente es informativo.	Fx, Fy, Fz

Por eso |Fh| aparece también sobre Fz aunque Fz no entre en la cuenta: la comparación entre la cizalla y la vertical es justamente el análisis interesante. Lo que nunca ocurre es que se dibuje sobre Momento o COP, que son de otra familia.

2. Qué hace cada fórmula

2.1 Fuerza resultante |F|

$$|F| = \sqrt{F_x^2 + F_y^2 + F_z^2}$$

Es la **magnitud del vector de fuerza**: el módulo de la fuerza total que la persona aplica sobre la plataforma, sin importar en qué dirección. Se obtiene aplicando el teorema de Pitágoras en tres dimensiones a las componentes.

Interpretación. En apoyo bipodal o durante la marcha, Fz (la vertical) domina ampliamente, de modo que |F| se parece muchísimo a Fz. Superponerla sobre cada componente muestra de un vistazo cuánto aporta cada eje al total: sobre Fz casi coinciden, sobre Fx se ve lo poco que aporta la cizalla.

Punto clave. |F| es **un solo valor por frame para toda la plataforma**, no uno por eje. Por eso, aunque se dibuje sobre tres gráficas, la curva es exactamente la misma en las tres, y el panel de resultados lo indica en lugar de repetir el número.

2.2 Fuerza horizontal |Fh|

$$|F_h| = \sqrt{F_x^2 + F_y^2}$$

Es la **fuerza de cizalla** en el plano de apoyo: la resultante de las dos componentes horizontales, que es la que frena e impulsa al sujeto. Queda separada de la vertical, que es la que sostiene el peso.

Interpretación. En marcha, |Fh| describe el ciclo de frenado y propulsión: crece en el contacto inicial (frenado) y vuelve a crecer en el despegue (propulsión). Es la base para estudios de tracción, resbalones y análisis de cambios de dirección.

Relación con |F|. Siempre se cumple $|F_h| \leq |F|$, y ambas coinciden solo si la componente vertical es nula. No necesita masa ni gravedad.

2.3 Fuerza normalizada (peso corporal)

$$F_{BW} = F / (masa \cdot g)$$

Divide la señal por el **peso** del sujeto (masa por gravedad), de modo que el resultado queda expresado en **pesos corporales (BW)**. Un valor de 1 BW significa que la fuerza equivale exactamente al peso de la persona.

Para qué sirve. Permite comparar sujetos de distinta masa: 900 N no significan lo mismo en alguien de 60 kg que en alguien de 100 kg, pero 1,5 BW sí es directamente comparable.

Por qué no dibuja curva. Dividir por una constante es un **reescalado puro**: la curva resultante tiene exactamente la misma forma que la original, solo que en otro eje. Dibujarla no aportaría información y solo ensuciaría la gráfica, así que la fórmula declara $salida = dato$ y sus valores van al recuadro de resultados. Igual se marca el pico con un punto sobre la señal visible, porque es el dato que más se busca.

Es la única de las tres que necesita un dato externo: la masa, que se carga en el panel izquierdo y se guarda por archivo en la base de datos. La gravedad es una constante de sesión (9,8 m/s² por defecto).

3. Las funciones de cálculo, línea por línea

Todas viven en `logica/formulas.py`, un módulo **sin ninguna dependencia de la interfaz**: recibe números y devuelve números. Eso es lo que permite testearlas sin abrir la aplicación.

3.1 `_componentes(*series)` — validación compartida

```
def _componentes(*series) -> list:
    arreglos = [np.asarray(serie, dtype=float) for serie in series]
    if len({len(arreglo) for arreglo in arreglos}) > 1:
        raise ErrorFormula(
            "Las componentes de fuerza deben tener la misma cantidad de muestras."
        )
    return arreglos
```

Línea	Qué hace
<code>arreglos = [np.asarray(serie, dtype=float) ...]</code>	Convierte cada componente a un arreglo de numpy de tipo float. Acepta listas, columnas de pandas o arreglos, y unifica el tipo para que las operaciones sean vectorizadas (rápidas, sin bucles de Python).
<code>if len({len(arreglo) ...}) > 1:</code>	Arma un conjunto con los largos. Si el conjunto tiene más de un elemento, alguna componente tiene distinta cantidad de muestras.
<code>raise ErrorFormula(...)</code>	Corta con un mensaje entendible en vez de dejar que numpy falle con un error de broadcasting incomprensible para el usuario.
<code>return arreglos</code>	Devuelve la lista ya validada y convertida.

Es una función privada (el guion bajo inicial lo indica) que centraliza la validación para no repetirla en cada fórmula.

3.2 `fuerza_resultante(fx, fy, fz)`

```
def fuerza_resultante(fx, fy, fz) -> np.ndarray:
    """|F| = sqrt(Fx² + Fy² + Fz²)."""
    fx, fy, fz = _componentes(fx, fy, fz)
    return np.sqrt(fx**2 + fy**2 + fz**2)
```

Línea	Qué hace
<code>fx, fy, fz = _componentes(fx, fy, fz)</code>	Valida los tres largos y convierte a numpy. Si no coinciden, sale por acá.
<code>return np.sqrt(fx**2 + fy**2 + fz**2)</code>	Eleva al cuadrado cada componente, las suma y saca la raíz. Todo elemento a elemento: el resultado es un vector con un valor por frame, del mismo largo que las entradas.

3.3 `fuerza_horizontal(fx, fy)`

```
def fuerza_horizontal(fx, fy) -> np.ndarray:
    """|Fh| = sqrt(Fx² + Fy²): la fuerza en el plano de apoyo."""
    fx, fy = _componentes(fx, fy)
    return np.sqrt(fx**2 + fy**2)
```

Línea	Qué hace
<code>fx, fy = _componentes(fx, fy)</code>	Misma validación, pero con dos componentes: Fz no participa.
<code>return np.sqrt(fx**2 + fy**2)</code>	Pitágoras en el plano horizontal. Al no incluir Fz, el resultado es siempre menor o igual que F .

3.4 El registro FUNCIONES_POR_COMPONENTES

```
FUNCIONES_POR_COMPONENTES.update({
    CLAVE_RESULTANTE: fuerza_resultante,
    CLAVE_HORIZONTAL: fuerza_horizontal,
})
```

Un diccionario que asocia la clave de cada fórmula con la función que la calcula. Gracias a esto, la interfaz **no conoce ninguna fórmula por nombre**: lee qué componentes pide (requiere_ejes), las busca en el mapeo, y llama a la función correspondiente. Agregar una cuarta fórmula de componentes no obliga a tocar una sola línea de la capa visual.

3.5 normalizar_por_peso(fuerza, masa, gravedad)

```
def normalizar_por_peso(fuerza, masa, gravedad) -> np.ndarray:
    """F / (m·g): expresa la fuerza en pesos corporales (BW)."""
    masa = float(masa or 0)
    gravedad = float(gravedad or 0)
    if masa <= 0:
        raise ErrorFormula(
            "Cargá la masa del sujeto en el panel izquierdo para normalizar por peso."
        )
    if gravedad <= 0:
        raise ErrorFormula("La gravedad debe ser mayor que cero.")
    return np.asarray(fuerza, dtype=float) / (masa * gravedad)
```

Línea	Qué hace
<code>masa = float(masa or 0)</code>	El <code>or 0</code> cubre el caso en que la masa venga como <code>None</code> (todavía no cargada): la convierte en 0 para que caiga en la validación de abajo en vez de reventar en el <code>float()</code> .
<code>if masa <= 0: raise ...</code>	Sin masa no hay peso, y dividir por cero daría infinito. En vez de eso, un mensaje que dice exactamente qué falta y dónde cargarlo.
<code>if gravedad <= 0: raise ...</code>	Misma protección para la gravedad, que es editable por el usuario.
<code>return np.asarray(...) / (masa*gravedad)</code>	El denominador $\text{masa} \cdot g$ es el peso en newtons. Dividir la señal por él da el resultado en pesos corporales.

3.6 resumen(x, y) — los valores que ve el usuario

Esta función no calcula ninguna fórmula: **describe** la curva que ya se calculó. Es la que alimenta el recuadro del panel, la caja que aparece al pasar el mouse y la tabla del popup.

```
def resumen(x, y) -> dict:
    x = np.asarray(x, dtype=float)
    y = np.asarray(y, dtype=float)
    mascara = np.isfinite(x) & np.isfinite(y)

    if not mascara.any():
        return {"pico": None, "x_pico": None, "minimo": None,
                "x_minimo": None, "media": None, "rms": None, "muestras": 0}

    x_validos = x[mascara]
    y_validos = y[mascara]
    indice_pico = int(np.argmax(y_validos))
    indice_minimo = int(np.argmin(y_validos))

    return {
        "pico": float(y_validos[indice_pico]),
        "x_pico": float(x_validos[indice_pico]),
        "minimo": float(y_validos[indice_minimo]),
        "x_minimo": float(x_validos[indice_minimo]),
        "media": float(np.mean(y_validos)),
        "rms": float(np.sqrt(np.mean(y_validos**2))),
        "muestras": int(y_validos.size),
    }
```

Línea	Qué hace
<code>mascara = np.isfinite(x) & np.isfinite(y)</code>	Marca las muestras válidas: descarta NaN e infinitos, que aparecen en huecos de captura o si una fórmula dividió por cero en algún frame.
<code>if not mascara.any(): return {...None...}</code>	Si no quedó ninguna muestra válida devuelve el diccionario con todos los campos en None en lugar de fallar. La interfaz muestra un guion.
<code>indice_pico = int(np.argmax(y_validos))</code>	Posición del valor máximo <i>dentro de las muestras válidas</i> .
<code>"x_pico": float(x_validos[indice_pico])</code>	El frame donde ocurre el pico. Es lo que permite marcar el punto en la gráfica y escribir «Pico (Frame 1720)».
<code>"media": float(np.mean(y_validos))</code>	Promedio aritmético de la curva.
<code>"rms": np.sqrt(np.mean(y_validos**2))</code>	Valor cuadrático medio. A diferencia de la media, no se cancela con valores negativos, así que describe la magnitud típica de la señal.
<code>"muestras": int(y_validos.size)</code>	Cuántas muestras válidas entraron. Sirve para saber si se descartó mucho.

3.7 formatear_valor(valor) — presentación

```
def formatear_valor(valor) -> str:
    if valor is None:
        return "-"
    valor = float(valor)
    if not math.isfinite(valor):
        return "-"

    magnitud = abs(valor)
    if magnitud >= 1000:
        texto = f"{valor:,.1f}"
    elif magnitud >= 1:
        texto = f"{valor:,.2f}"
    elif magnitud > 0:
        texto = f"{valor:.4g}"
    else:
        return "0"
    return texto.replace(",", " ")
```

Existe por un motivo concreto: el formato %g de Python convierte 1138,2 en 1.14e+03, que es justo el rango de los valores de fuerza más habituales y lo último que quiere leer alguien mirando un pico. Esta función elige la cantidad de decimales según la magnitud y nunca usa notación científica.

Entrada	Regla aplicada	Salida
1138.02	$\geq 1000 \rightarrow$ un decimal y separador de miles	1 138.0
12.5	$\geq 1 \rightarrow$ dos decimales	12.50
0.18085	$< 1 \rightarrow$ cuatro cifras significativas	0.1809
0	caso especial	0
None o NaN	sin dato	—

4. Fórmulas propias (las que escribe el usuario)

Además de las tres generales, el popup «Más fórmulas...» permite escribir una expresión libre. Esto es delicado: ejecutar texto que escribe el usuario es la vía clásica de ejecución de código arbitrario. Por eso **no se usa** `eval()` de **Python** en ningún momento.

4.1 `_normalizar_expresion(expresion)`

```
return texto.replace("^", "**").replace(".", "").replace("x", "*").replace(",", "", ".")
```

Traduce la notación que la gente escribe naturalmente a la sintaxis de Python: el acento circunflejo como potencia, el punto medio y la cruz como multiplicación, y la coma como separador decimal. Así `sqrt(Fx^2 + Fy^2)` funciona sin que el usuario tenga que saber que Python usa `**`.

4.2 `nombres_usados(expresion)`

Recorre el árbol sintáctico y devuelve el conjunto de variables mencionadas, sin las funciones ni las constantes. Tiene dos usos:

Uso	Para qué
Validación en vivo	Mientras se escribe, avisa «No conozco: Fq» sin llegar a calcular nada. El botón Aplicar queda deshabilitado hasta que la fórmula sea válida.
Decidir el alcance	Si la expresión menciona F o <code>senal</code> , es una fórmula por señal y se aplica a todas las gráficas visibles. Si en cambio nombra columnas concretas, es una fórmula entre señales : da una sola curva y se dibuja únicamente sobre esas señales.

4.3 `evaluar_expresion(expresion, variables)` — el evaluador seguro

Analiza la expresión con `ast.parse(texto, mode="eval")`, que produce un árbol sintáctico **sin ejecutar nada**. Después una función recursiva recorre ese árbol y solo acepta cinco tipos de nodo:

Nodo permitido	Qué se acepta	Qué se rechaza
Constant	Números enteros y decimales.	Textos, booleanos.
Name	Las variables entregadas (Fx, Fy, masa, g...) y las constantes pi y e.	Cualquier otro nombre: error indicando cuál.
BinOp	Suma, resta, multiplicación, división, potencia y módulo.	Operadores de bits, comparaciones.
UnaryOp	Signo positivo y negativo.	Negación lógica.
Call	Solo las funciones de la lista blanca, y solo con argumentos posicionales.	Cualquier otra llamada; argumentos con nombre.

Todo lo demás —acceso a atributos, índices, comprensiones de lista, asignaciones— cae en el `raise final`. Estas expresiones fallan de forma controlada, y hay un test que lo verifica explícitamente:

```
__import__('os').system('echo hola') -> rechazada
open('x.txt', 'w') -> rechazada
Fx.__class__ -> rechazada
[f for f in Fx] -> rechazada
```


(1).__class__.__bases__	-> rechazada
exec('x=1')	-> rechazada

Importante para quien mantenga el código: no sustituir este evaluador por `eval()` ni por bibliotecas como `numexpr` «porque es más simple». La lista blanca es la única barrera entre el cuadro de texto y el intérprete.

4.4 `_como_vector(resultado, variables)`

Una fórmula siempre tiene que devolver un vector del largo de la señal, pero una expresión como `masa * g` da un único número. Esta función mide el largo de las variables que sí son vectores y, si el resultado es escalar, lo repite para completar ese largo. Además verifica que quede al menos un valor finito: si toda la curva salió NaN o infinito, avisa que revisen las divisiones por cero.

5. Qué información viaja hacia la gráfica

El módulo de fórmulas solo devuelve números. Quien los reparte es AreaCentralGraficas. Este es el recorrido completo:

Pas o	Método	Qué hace
1	aplicar_formula()	Punto de entrada. Determina las gráficas visibles y llama al cálculo.
2	_calcular_formula()	Busca las componentes en el mapeo, llama a la función de FUNCIONES_POR_COMPONENTES y arma una fila por señal destino.
3	_fila_resultado()	Por cada señal: recorta las muestras válidas, calcula el resumen() y anota la procedencia de los datos.
4	set_curva_formula()	En cada gráfica: dibuja la curva (si corresponde), elige el eje y crea el marcador del pico.
5	resultadosFormulaCambieron	Señal de Qt que lleva los números al panel derecho y al popup.

5.1 El paquete de datos que se emite

```
{
  "nombre": "Fuerza horizontal |Fh|",
  "expresion": "sqrt(Fx^2 + Fy^2)",
  "unidad": "N",
  "dibuja_curva": True,
  "resultados": [
    {
      "columna": "Fx",
      "senal": "Fuerza X - Fx",
      "resumen": {"pico": 166.17, "x_pico": 4696.0, "minimo": 64.99,
                  "media": 105.09, "rms": 107.35, "muestras": 6165},
      "fuente": "filtrada",
      "detalle_filtro": "un filtro por debajo de 15 Hz",
    },
    ...
  ],
}
```

Línea	Qué hace
nombre / expresion / unidad	Encabezan el recuadro de resultados y rotulan el eje de la curva.
dibuja_curva	Viene de salida. En False (normalizada) no se traza ninguna curva: solo los números y el punto del pico.
columna / senal	Identificador interno y nombre legible de la señal donde se dibuja.
resumen	El diccionario de la sección 3.6. Alimenta el recuadro del panel, la caja del mouse y la tabla del popup.
fuente	«original», «filtrada» o «mixta». Describe los datos que entraron al cálculo , no la gráfica donde se pinta: si Fh se calculó con Fx y Fy filtradas, la fila de Fz también dice «filtrada».

Línea	Qué hace
detalle_filtro	El corte concreto que se usó, por ejemplo «un filtro por debajo de 15 Hz». Queda registrado para poder reproducir el análisis.

5.2 Qué dibuja cada elemento

Elemento	Qué es	Regla
Curva de fórmula	Una curva más superpuesta, en verde (violeta en modo daltónico).	Nunca reemplaza la original ni la filtrada, y siempre queda por encima de ambas.
Eje derecho	Un segundo eje Y propio para la curva.	Se activa cuando la escala no es comparable: si al compartir eje alguna curva ocupara menos del 35 % de la altura, se separa. Por eso F comparte eje con Fz pero no con Fx.
Marcador del pico	Un punto sobre el máximo de la fórmula.	Si hay curva, se apoya en ella. Si no la hay, se apoya sobre la señal visible en el frame donde la fórmula alcanza su máximo.
Caja de valores	Recuadro con pico, mínimo, media y RMS.	Aparece solo mientras el cursor está encima del pico o de la curva, y se ubica del lado donde entra completa.

6. Cómo agregar una fórmula nueva

Si la fórmula se construye a partir de las componentes de fuerza, alcanza con tocar **dos lugares del mismo archivo**. La interfaz se adapta sola: el desplegable, el popup, el selector de fuente, el marcador del pico y el botón Quitar ya funcionan.

Paso 1 — la función de cálculo

```
def angulo_vertical(fx, fy, fz) -> np.ndarray:
    """Inclinación del vector de fuerza respecto de la vertical, en grados."""
    fx, fy, fz = _componentes(fx, fy, fz)
    return np.degrees(np.arctan2(np.sqrt(fx**2 + fy**2), fz))
```

Paso 2 — declararla en FORMULAS_GENERALES

```
{
    "clave": "angulo",
    "nombre": "Ángulo respecto de la vertical",
    "expresion": "atan2(sqrt(Fx^2 + Fy^2), Fz)",
    "descripcion": "Cuánto se aparta de la vertical el vector de fuerza.",
    "unidad": "grados",
    "requiere_ejes": ("Fx", "Fy", "Fz"),
    "requiere_masa": False,
    "mostrar_en": ("Fx", "Fy", "Fz"),
    "salida": SALIDA_CURVA,
}
```

```
FUNCIONES_POR_COMPONENTES["angulo"] = angulo_vertical
```

Qué significa cada campo

Campo	Para qué sirve
clave	Identificador interno. Une la entrada con su función.
nombre	Lo que se lee en el desplegable y en la leyenda.
expresion	Se muestra como ayuda debajo del desplegable.
descripcion	Texto explicativo en el panel y en el tooltip.
unidad	Rotula el eje y acompaña cada valor. Además decide si la curva necesita eje propio.
requiere_ejes	Componentes necesarias. Si falta alguna, la aplicación avisa cuál en lugar de fallar.
requiere_masa	Si la fórmula depende de la masa del sujeto.
mostrar_en	Sobre qué gráficas se dibuja. Si se omite, se usan las mismas de requiere_ejes.
salida	SALIDA_CURVA dibuja la curva; SALIDA_DATO solo muestra los valores.

Para fórmulas que no salen de las componentes (por ejemplo, una que derive la señal) hay que agregar además su rama en `_calcular_formula()`, y ahí sí conviene declarar que necesita filtro previo, porque derivar amplifica el ruido.

Cobertura de tests

Las fórmulas están cubiertas por `tests/test_logica.py`, que se ejecuta con:

```
python -m unittest tests.test_logica
```

Incluye la verificación de que $|F_h|$ nunca supera a $|F|$, que la normalizada conserva la forma de la señal (la justificación de que no se dibuje), el formateo de valores y, sobre todo, que el evaluador de expresiones rechaza el código arbitrario.